

Apollo AgriVerse

Engineering an Intelligent Agricultural Ecosystem via Weather API Architecture

MindForgeAI Internship Program | Computational Engineering & Smart Systems | Target Scope: Live Agricultural Weather Microservice (India)

1. Executive Overview & System Philosophy

Apollo AgriVerse is a unified technological ecosystem engineered to model, predict, and continuously optimize agricultural operations across the crop lifecycle. Traditional precision agriculture operates reactively—monitoring static sensor parameters or relying on historical averages, taking action only after resource degradation has occurred.

The **Weather API Microservice** replaces delayed observation loops with a live meteorological stream that dynamically feeds real-time telemetry—such as ambient temperature, relative humidity, precipitation, wind speed, and solar radiation—directly into downstream digital twin crop models. Engineered specifically for Indian agricultural topography, the API incorporates geographic boundary validation, automated data sanitization, in-memory response caching, and a bulletproof 3-tier hybrid fallback routing architecture.

Fundamental Concept Glossary

- **Hybrid Fallback Engine:** A multi-layered request routing mechanism that prioritizes real-time live APIs and gracefully degrades to satellite data or offline local datasets during network outages.
- **Telemetry:** Automated, continuous collection and transmission of environmental parameters from remote weather stations or satellites.
- **In-Memory TTL Cache:** A high-speed memory layer that stores recent API responses for a set Time-To-Live (15 minutes), serving identical location queries in under 5 ms .
- **Data Sanitization:** Preprocessing logic that intercepts missing or corrupted telemetry values (e.g., NASA's -999.0 fill codes) and normalizes them into valid numerical bounds.
- **OpenAPI / Swagger UI:** Auto-generated interactive API documentation built on Pydantic schemas, enabling visual API testing and formal contract enforcement.

2. Implemented Weather Architecture & Data Sources

The Weather API Microservice relies on three synchronized data tiers to guarantee high availability and sub-second response times.

2.1 Multi-Tier Weather Data Matrix

Data Tier	Source / Endpoint	Update Frequency	Key Parameters Captured	Operational Role in Apollo AgriVerse
Tier 1 (Primary)	Open-Meteo API	Live (Every 15 mins)	Air Temp (°C), Humidity (%), Rainfall (mm), Wind Speed (m/s), Solar Radiation (W/m²)	Serves 100% true live weather telemetry for real-time digital twin state updates.
Tier 2 (Secondary)	NASA POWER AG API	Daily Snapshot (~3-day satellite lag)	Temperature (T_{2M}), Relative Humidity (RH_{2M}), Precipitation ($PRECOTCORR$), Wind Speed (WS_{2M}), Solar Radiation ($ALLSKY_SFC_SW_DWN$)	Acts as secondary online satellite backup with specialized agricultural solar parameters.
Tier 3 (Tertiary Failsafe)	<code>weather_raw_dataset.csv</code>	Static Offline Dataset	Latitude, Longitude, Temp (°C), Humidity (%), Precip (mm), Pressure (hPa), Wind (km/h), Solar (W/m²)	Guarantees offline availability by computing nearest geographic point via coordinate distance algorithm.

3. The Three Core Pillars of the Weather API

- Real-Time Telemetry Stream (Open-Meteo Engine):** Fetches live microclimate metrics without requiring authentication keys, updating every 15 minutes to reflect atmospheric changes.
- Satellite & Offline Resilient Routing (NASA POWER & CSV Engine):** Intercepts network timeouts or socket failures. If Open-Meteo is unreachable, requests route seamlessly to NASA POWER AG; if all internet access drops, the system matches user coordinates against `weather_raw_dataset.csv` using Manhattan distance approximation:
$$d = |\text{latitude}\{\text{CSV}\} - \text{latitude}\{\text{User}\}| + |\text{longitude}\{\text{CSV}\} - \text{longitude}\{\text{User}\}|$$
- Automated Bounding & Sanitization:** Enforces Indian geographic boundary constraints ($6.0^\circ\text{N} \leq \text{Latitude} \leq 37.5^\circ\text{N}$ and $68.0^\circ\text{E} \leq \text{Longitude} \leq 97.5^\circ\text{E}$) and filters invalid satellite flags (converting -999.0 indicators to 0.0).

4. Performance Metrics & Caching Mechanics

To minimize latency and prevent rate-limiting when multiple requests query identical farm coordinates, the API implements spatial coordinate rounding and in-memory Time-To-Live (TTL) caching:

- **Cache Key Computation:** Coordinates are rounded to 2 decimal places ($\sim 1.1 \text{ km}$ spatial grid resolution).
- **TTL Duration:** 900 seconds (15 minutes).

Performance Benchmarks

- **Cached Hits:** $< 5 \text{ ms}$ response time.
- **Live Open-Meteo Queries:** $\sim 200 - 400 \text{ ms}$ response time.
- **NASA POWER Queries:** $\sim 1.5 - 2.5 \text{ s}$ response time.
- **CSV Offline Fallback:** $\sim 10 - 20 \text{ ms}$ execution time.

5. Technical Implementation & Microservices Endpoint Roadmap

- **Phase 1: Core Service Engineering (`05_Backend/api/weather_api/`)** * **What:** Developed module logic for NASA POWER (`nasa_api.py`), script testing (`test_nasa_weather.py`), and local dataset fallback processing (`weather_service.py`). * **How:** Utilized `requests` and `pandas` libraries to build dynamic date calculation routines (`datetime.utcnow() - timedelta(days=3)`) and distance-minimization logic.
- **Phase 2: REST API & Pydantic Validation (`05_Backend/app/main.py`)** * **What:** Built high-performance FastAPI microservice endpoints with CORS cross-origin middleware. * **How:** Implemented GET `/weather` endpoint accepting `latitude` and `longitude` query parameters. Enforced strict range constraints using Pydantic `Field` and `Query` schemas.
- **Phase 3: Live Verification & Testing** * **What:** Verified interactive OpenAPI documentation (`http://127.0.0.1:8000/docs`) and CLI terminal outputs. * **How:** Tested edge cases including boundary violation queries, simulated network drops, and live coordinates across major Indian agricultural zones (e.g., Solapur, Maharashtra at 17.66°N , 75.91°E).
- **Phase 4: Final Handoff & Frontend Integration** * **What:** Handed off clean, self-documenting JSON APIs for integration into the digital twin dashboard (`04_Frontend/`). * **How:** Created standardized response payloads with source tracking tags (`"Open-Meteo (Live)"` , `"NASA POWER AG"` , or `"Local CSV Fallback"`) for transparent frontend rendering.